

Title: Lottery Scheduler

Name: Kadiatou Diallo

Class: CIS 345

Date: 5/16/2026

Prof: Yanilda

Introduction

This project implements a Lottery Scheduler in the xv6 operating system. Lottery scheduling allocates CPU time probabilistically based on the number of tickets assigned to each process. Processes with more tickets receive more CPU time. The project required modifying the xv6 scheduler, implementing new system calls, tracking process statistics, and analyzing scheduler fairness using graphs.

Objectives: to Understand xv6 kernel structure, learn how CPU scheduling works, implement a Lottery Scheduler, add new system calls and Analyze scheduler fairness.

System Calls Implemented

settickets () The settickets () system call sets the number of tickets assigned to the calling process.

The getpinfo () system call returns scheduling statistics for all processes, including:

- process ID
- ticket count
- tick count
- process usage status

Kernel Modifications

The process structure was modified by adding two fields:

tickets: stores number of lottery tickets

ticks: stores how many times the process was scheduled

allocproc (): Default values were initialized when a process is created

fork (): Child processes inherit the same number of tickets as their parent
Scheduler Algorithm

The scheduler first calculates the total number of tickets for all runnable processes.

A random number is generated between 0 and the total number of tickets.

The scheduler iterates through all runnable processes and accumulates their ticket counts.

The process whose ticket range contains the winning ticket is selected to run.

Test Program

Three CPU-bound processes were created with ticket ratios 30:20:10. Each process continuously executed an infinite loop to compete for CPU time. The scheduler statistics were collected using `getpinfo ()`.

Result Analysis

The process with 30 tickets received the highest number of CPU ticks.
The process with 20 tickets received a moderate number of ticks.
The process with 10 tickets received the lowest number of ticks.

The results closely match the expected proportional distribution of CPU time, demonstrating that the Lottery Scheduler behaves correctly.

The graphs clearly show that processes with more tickets receive more CPU time. The CPU allocation follows the expected 3:2:1 ratio.

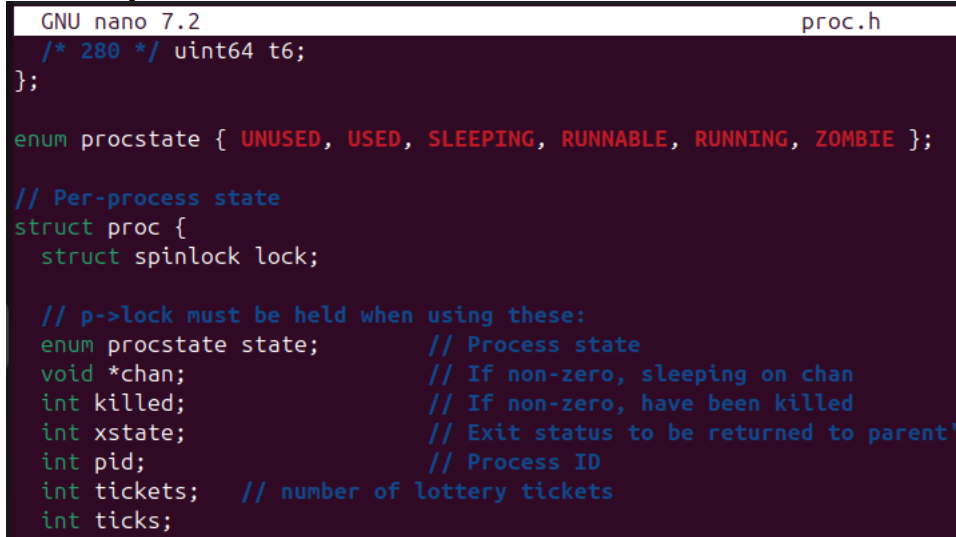
Conclusion

The Lottery Scheduler was successfully implemented in xv6. The scheduler correctly allocated CPU time proportionally according to the number of tickets assigned to each process. Experimental results and graphs

verified the fairness and correctness of the scheduler implementation.

Screenshot

Kernel/proc.h

A screenshot of a text editor window showing the kernel/proc.h file. The editor's title bar indicates 'GNU nano 7.2' and the file name 'proc.h'. The code defines a 280-byte array 't6', an enum 'procstate' with values UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, and ZOMBIE, and a 'struct proc' containing a spinlock, process state, channel, kill status, exit status, parent ID, lottery tickets, and ticks.

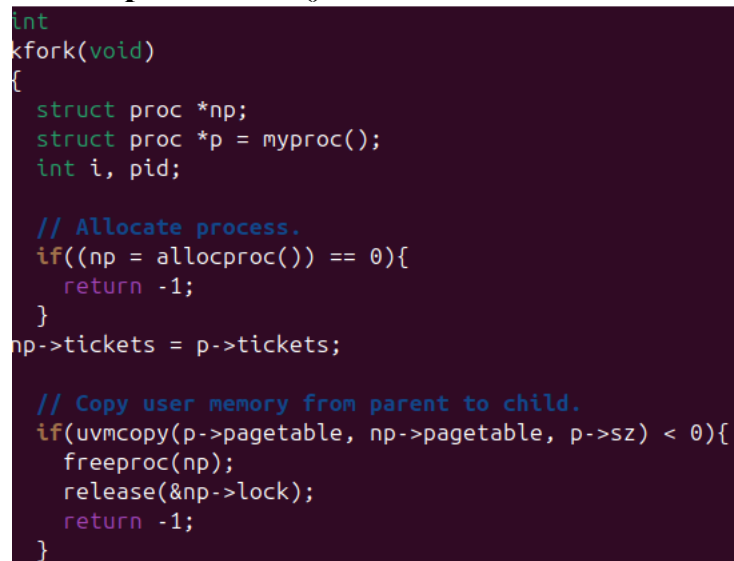
```
GNU nano 7.2 proc.h
/* 280 */ uint64 t6;
};

enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

// Per-process state
struct proc {
    struct spinlock lock;

    // p->lock must be held when using these:
    enum procstate state;      // Process state
    void *chan;               // If non-zero, sleeping on chan
    int killed;               // If non-zero, have been killed
    int xstate;               // Exit status to be returned to parent
    int pid;                  // Process ID
    int tickets;              // number of lottery tickets
    int ticks;
```

kernel/proc.c fork()

A screenshot of a text editor window showing the 'fork()' function in kernel/proc.c. The function takes a 'void' argument and returns an 'int'. It allocates a new process structure 'np', copies the parent's state to it, and returns the pointer to the new process.

```
int
fork(void)
{
    struct proc *np;
    struct proc *p = myproc();
    int i, pid;

    // Allocate process.
    if((np = allocproc()) == 0){
        return -1;
    }
    np->tickets = p->tickets;

    // Copy user memory from parent to child.
    if(uvmcopy(p->pagetable, np->pagetable, p->sz) < 0){
        freeproc(np);
        release(&np->lock);
        return -1;
    }
```

kernel/proc.c allocproc

```

GNU nano 7.2                                     proc.c
// If there are no free procs, or a memory allocation fails, return
struct proc*
allocproc(void)
{
    struct proc *p;

    for(p = proc; p < &proc[NPROC]; p++) {
        acquire(&p->lock);
        if(p->state == UNUSED) {
            goto found;
        } else {
            release(&p->lock);
        }
    }
    return 0;

found:
    p->state = USED;
    p->pid = nextpid++;
    p->tickets = 1;    // default
    p->ticks = 0;
    // Allocate a trapframe page.
    if((p->trapframe = (struct trapframe *)kalloc()) == 0){
        freeproc(p);
        release(&p->lock);
        return 0;
    }
}

```

Pstat.h

```

GNU nano 7.2                                     pstat.h
#ifndef _PSTAT_H_
#define _PSTAT_H_

#include "param.h"

struct pstat {
    int inuse[NPROC];
    int tickets[NPROC];
    int pid[NPROC];
    int ticks[NPROC];
};

#endif

```

scheduler

```

void
scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();

    c->proc = 0;

    for(;;){
        intr_on();
        int total_tickets = 0;

        // Count total runnable tickets
        for(p = proc; p < &proc[NPROC]; p++){
            acquire(&p->lock);
            if(p->state == RUNNABLE)
                total_tickets += p->tickets;

            release(&p->lock);
        }

        // Pick winning ticket
        int winning_ticket = 0;
        if(total_tickets > 0)
            winning_ticket = rand() % total_tickets;

        int ticket_sum = 0;

```

Kernel/sysproc.c

```

GNU nano 7.2 sysproc.c

uint64
sys_settickets(void)
{
    int n;
    argint(0, &n);
    return -1;
    if( n <= 0)
        return -1;
    myproc()->tickets = n;
    return 0;
}

uint64
sys_getpinfo(void)
{
    uint64 addr;
    struct pstat st;
    struct proc *p;
    argaddr(0, &addr);
    return -1;
    for(int i = 0; i < NPROC; i++) {
        p = &proc[i];
        st.inuse[i] = (p->state != UNUSED);
        st.pid[i] = p->pid;
        st.tickets[i] = p->tickets;
        st.ticks[i] = p->ticks;
    }
}

```

kernel/syscall.h

```
GNU nano 7.2                                     syscall.h
#define SYS_pipe      4
#define SYS_read      5
#define SYS_kill      6
#define SYS_exec      7
#define SYS_fstat     8
#define SYS_chdir     9
#define SYS_dup      10
#define SYS_getpid    11
#define SYS_sbrk     12
#define SYS_pause     13
#define SYS_uptime    14
#define SYS_open     15
#define SYS_write     16
#define SYS_mknod     17
#define SYS_unlink    18
#define SYS_link      19
#define SYS_mkdir     20
#define SYS_close     21
#define SYS_getprocs  22
#define SYS_sem_wait  23
#define SYS_sem_signal 24
#define SYS_testsem   25
#define SYS_sleep     13
#define SYS_settickets 26
#define SYS_getpinfo  27
```

Syscall.c

```
GNU nano 7.2                                     syscall.c
extern uint64 sys_sem_signal(void);
extern uint64 sys_testsem(void);
extern uint64 sys_settickets(void);
extern uint64 sys_getpinfo(void);
[SYS_open]    sys_open,
[SYS_write]   sys_write,
[SYS_mknod]   sys_mknod,
[SYS_unlink]  sys_unlink,
[SYS_link]    sys_link,
[SYS_mkdir]   sys_mkdir,
[SYS_close]   sys_close,
[SYS_getprocs] sys_getprocs,
[SYS_sem_wait] sys_sem_wait,
[SYS_sem_signal] sys_sem_signal,
[SYS_testsem] sys_testsem,
[SYS_settickets] sys_settickets,
[SYS_getpinfo] sys_getpinfo,
};
```

Lotterytest

```
GNU nano 7.2 user/lotterytest.c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "kernel/pstat.h"
#include "user/user.h"
int
main(void)
{
    if(fork() == 0)
    {
        settickets(30);
        while(1){

        }
    }
    if(fork() == 0)
    {
        settickets(20);
        while(1){

        }
    }
    if(fork() == 0)
    {
        settickets(10);
        while(1){

        }
    }
}
```

User.h

```
GNU nano 7.2 user/user.h
int stat(const char*, struct stat*);
char* strcpy(char*, const char*);
void *memmove(void*, const void*, int);
char* strchr(const char*, char c);
int strcmp(const char*, const char*);
char* gets(char*, int max);
uint strlen(const char*);
void* memset(void*, int, uint);
int atoi(const char*);
int memcmp(const void *, const void *, uint);
void *memcpy(void *, const void *, uint);
char* sbrk(int);
char* sbrklazy(int);
struct semaphore;
int sem_init(struct semaphore*, int);
void sem_wait(struct semaphore*);
void sem_signal(struct semaphore*);
int sleep(int);
int settickets(int);
int getpinfo(struct pstat *);

// printf.c
void fprintf(int, const char*, ...) __attribute__((format (printf, 2, 3)));
void printf(const char*, ...) __attribute__((format (printf, 1, 2)));
```

Usys

```
GNU nano 7.2 user/usys.pl
entry("exit");
entry("wait");
entry("pipe");
entry("read");
entry("write");
entry("close");
entry("kill");
entry("exec");
entry("open");
entry("mknod");
entry("unlink");
entry("fstat");
entry("link");
entry("mkdir");
entry("chdir");
entry("dup");
entry("getpid");
entry("sbrk");
entry("pause");
entry("uptime");
entry("getprocs");
entry("sem_wait");
entry("sem_signal");
entry("testsem");
entry("sleep");
entry("settickets");
entry("getpinfo")
```

```
GNU nano 7.2
{
    settickets(20);
    while(1){

    }
}
if(fork() == 0)
{
    settickets(10);
    while(1){

    }
}
struct pstat st;
getpinfo(&st);
printf("PID\tTickets\tTicks\n");
for( int i = 0; i<NPROC; i++){
if(st.inuse[i]== 1 && st.tickets[i] > 0) {
printf("%d\t%d\t%d\n",
        st.pid[i],
        st.tickets[i],
        st.ticks[i]);
}
}

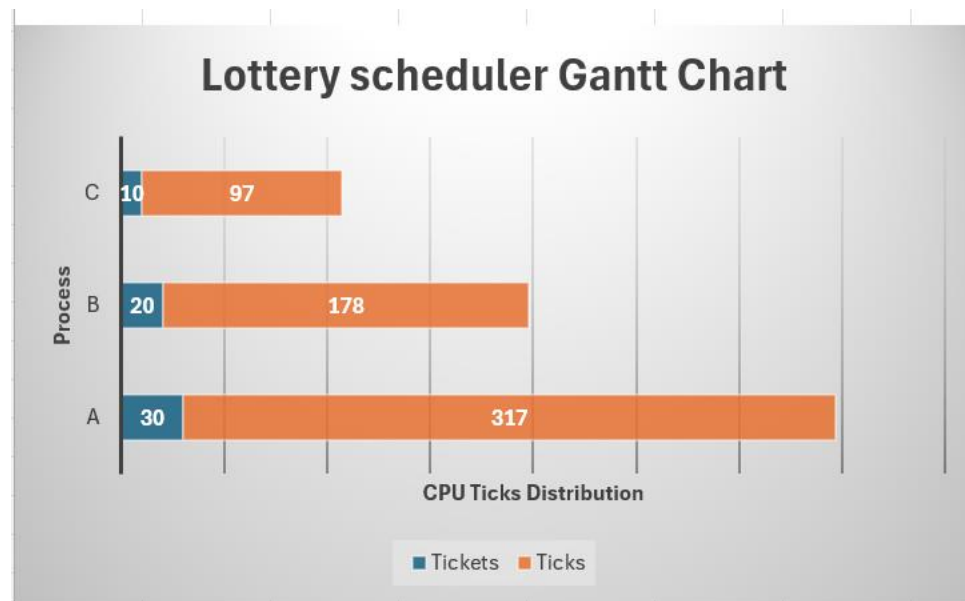
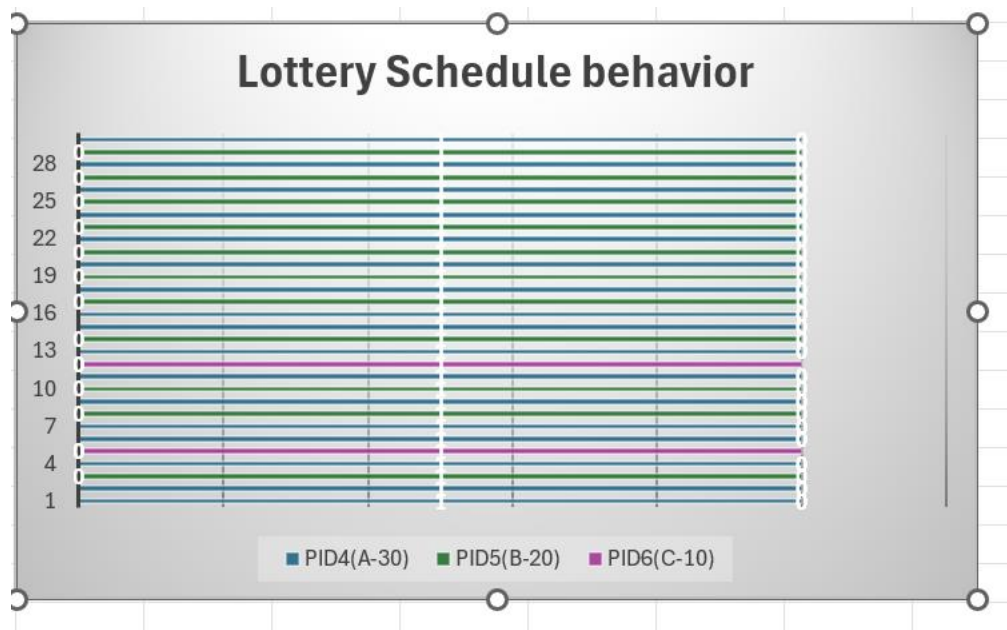
exit(0);
}
```


Makefile

```
GNU nano 7.2 Makefile
UPROGS=\
    $U/_cat\
    $U/_echo\
    $U/_forktest\
    $U/_grep\
    $U/_init\
    $U/_kill\
    $U/_ln\
    $U/_ls\
    $U/_mkdir\
    $U/_rm\
    $U/_sh\
    $U/_stressfs\
    $U/_usertests\
    $U/_grind\
    $U/_wc\
    $U/_zombie\
    $U/_logstress\
    $U/_forphan\
    $U/_dorphan\
    $U/_getprocs\
    $U/_qlocktest\
    $U/_semtest\
    $U/_lotterytest\
```

```
tick:304 pid:4 tickets:30
tick:93 pid:6 tickets:10
tick:305 pid:4 tickets:30
tick:306 pid:4 tickets:30
tick:169 pid:5 tickets:20
tick:170 pid:5 tickets:20
tick:171 pid:5 tickets:20
tick:94 pid:6 tickets:10
tick:172 pid:5 tickets:20
tick:173 pid:5 tickets:20
tick:307 pid:4 tickets:30
tick:95 pid:6 tickets:10
tick:308 pid:4 tickets:30
tick:309 pid:4 tickets:30
tick:96 pid:6 tickets:10
tick:310 pid:4 tickets:30
tick:174 pid:5 tickets:20
tick:311 pid:4 tickets:30
tick:312 pid:4 tickets:30
tick:313 pid:4 tickets:30
tick:314 pid:4 tickets:30
tick:315 pid:4 tickets:30
tick:175 pid:5 tickets:20
tick:176 pid:5 tickets:20
tick:177 pid:5 tickets:20
tick:97 pid:6 tickets:10
tick:316 pid:4 tickets:30
tick:178 pid:5 tickets:20
tick:317 pid:4 tickets:30
PID      Tickets Ticks
4         30     317
5         20     178
```

Graph



Three processes were executed using a lottery scheduler with ticket ratios 30, 20, and 10. The scheduler selected processes according to their ticket probability. The results showed that the process with 30 tickets received the highest CPU time (317 ticks), the process with 20 tickets received moderate CPU time (178 ticks), and the process with 10 tickets received the least CPU time (97 ticks). This demonstrates that the lottery scheduler distributes CPU time proportionally to the number of tickets assigned to each process.